

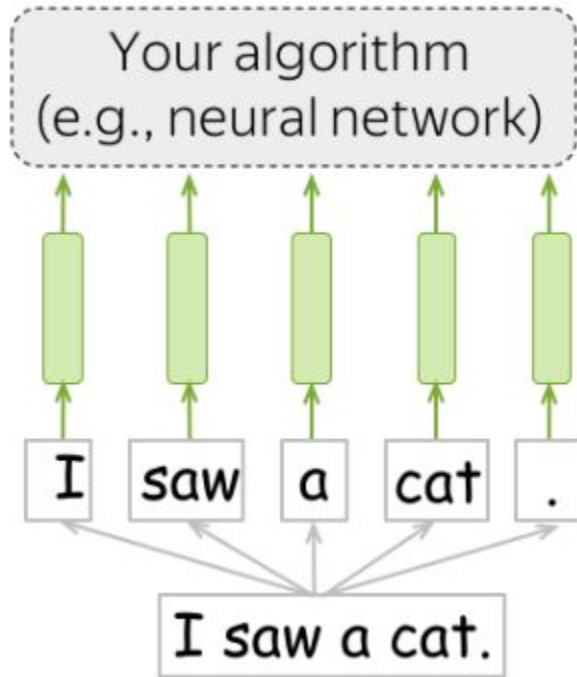
Embeddings

Mark Blumenau

What you probably already know

- 1) Tokenization
- 2) Bag of Words
- 3) TF-IDF
- 4) Stemming
- 5) Lemmatization

What we will learn today



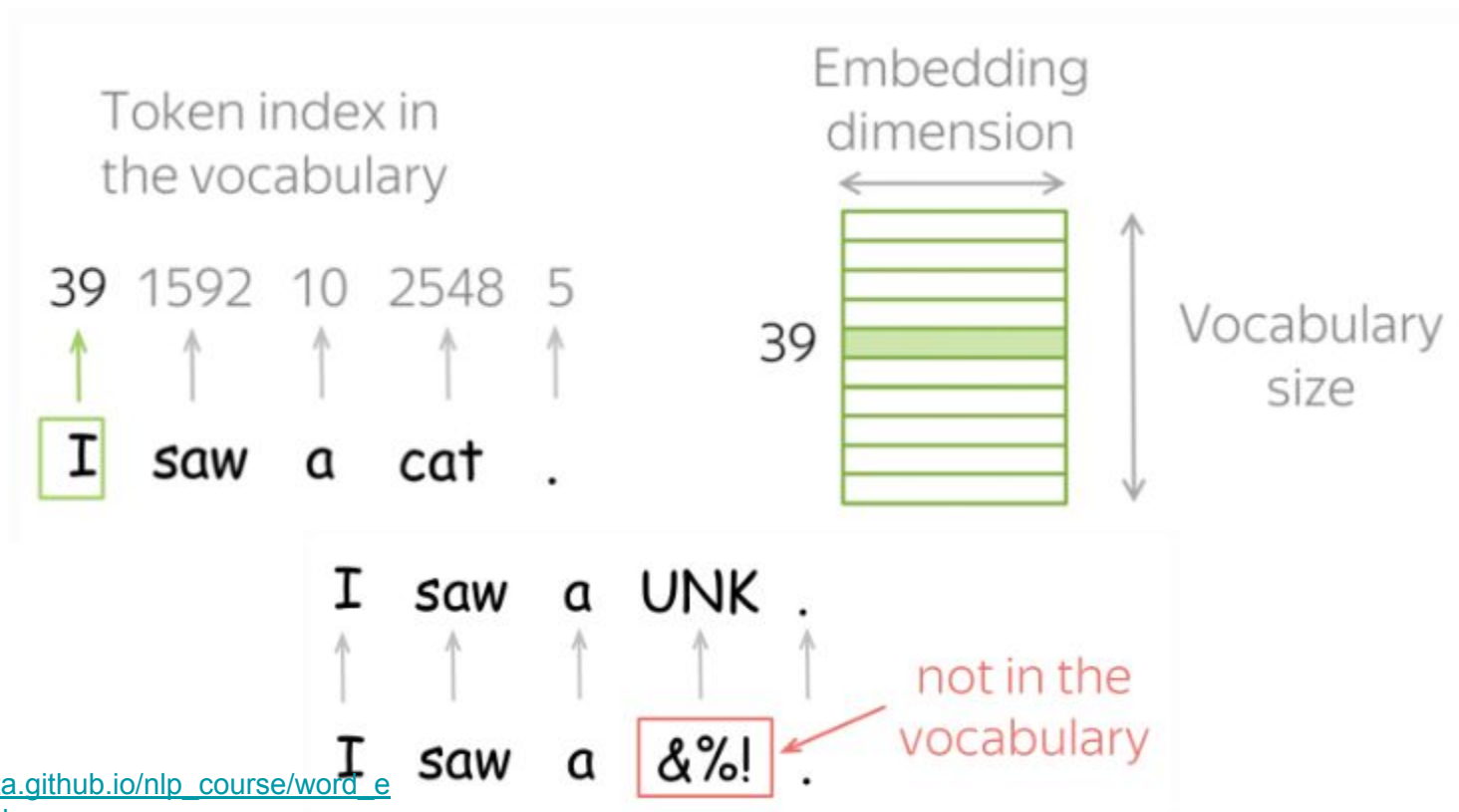
Any algorithm for solving a task

Word representation - vector
(input for your model/algorithm)

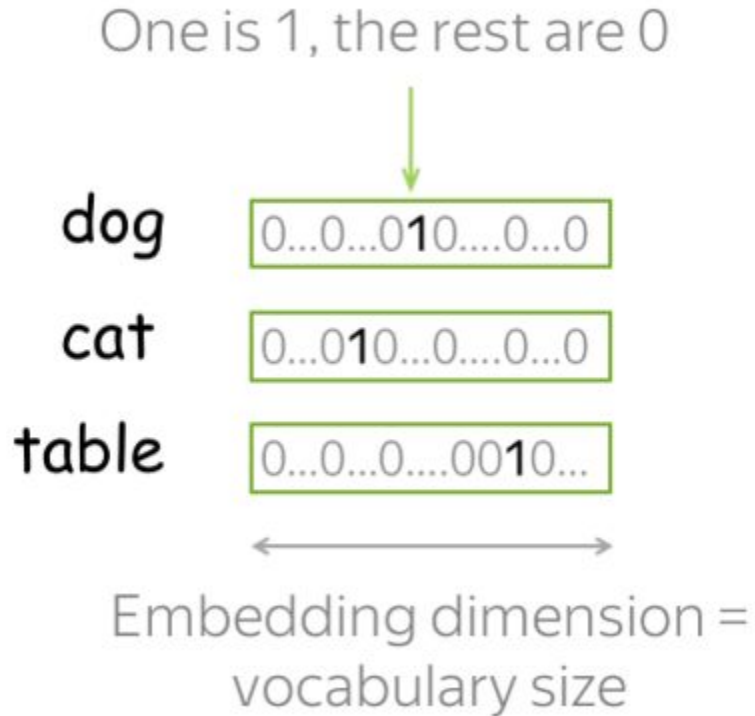
Sequence of tokens

Text (your input)

Tokenization



One hot encoding is bad



What is an embedding?

Now look how this word is used in different contexts:

A bottle of **tezgüino** is on the table.

Everyone likes **tezgüino**.

Tezgüino makes you drunk.

We make **tezgüino** out of corn.

Can you understand what **tezgüino** means ?



Let's use context

A bottle of **tezgüino** is on the table.

Everyone likes **tezgüino**.

Tezgüino makes you drunk.

We make **tezgüino** out of corn.



Tezgüino is a kind of alcoholic beverage made from corn.

With context, you can understand the meaning!

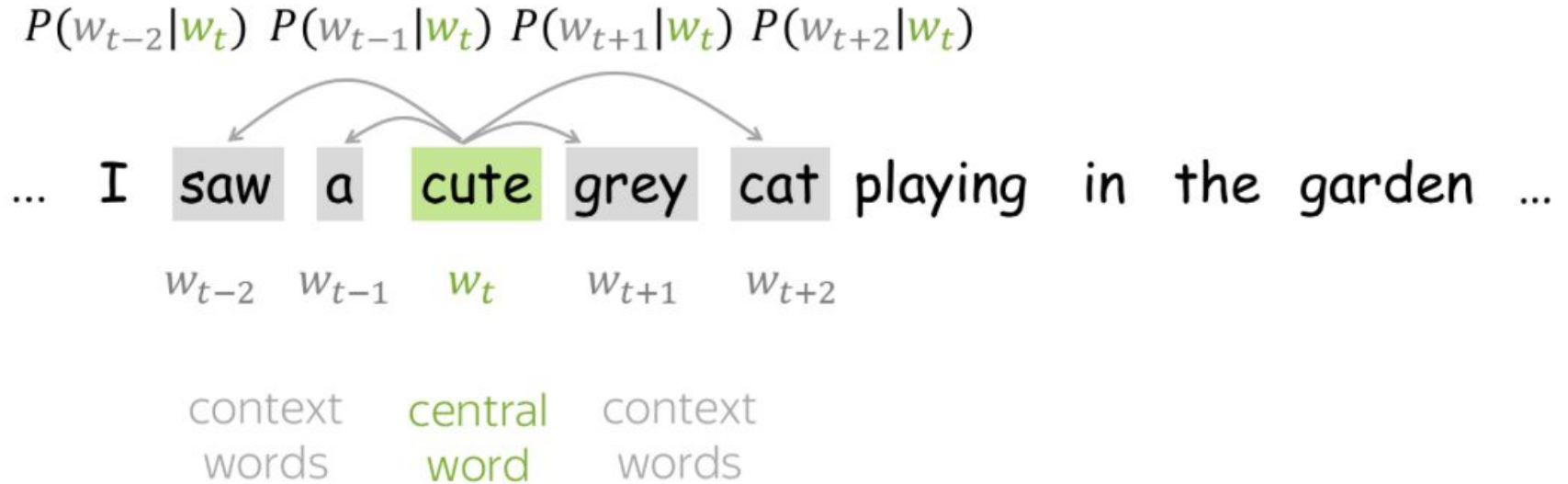


If you know russian or any slavic language, a real-time example of your neural network running:



[watch](#)

Word2Vec



Word2Vec

$$\text{Loss} = J(\theta) = -\frac{1}{T} \log L(\theta) = -\frac{1}{T} \sum_{t=1}^T \sum_{\substack{-m \leq j \leq m, \\ j \neq 0}} \log P(w_{t+j} | w_t, \theta)$$

agrees with our
plan above



go over text



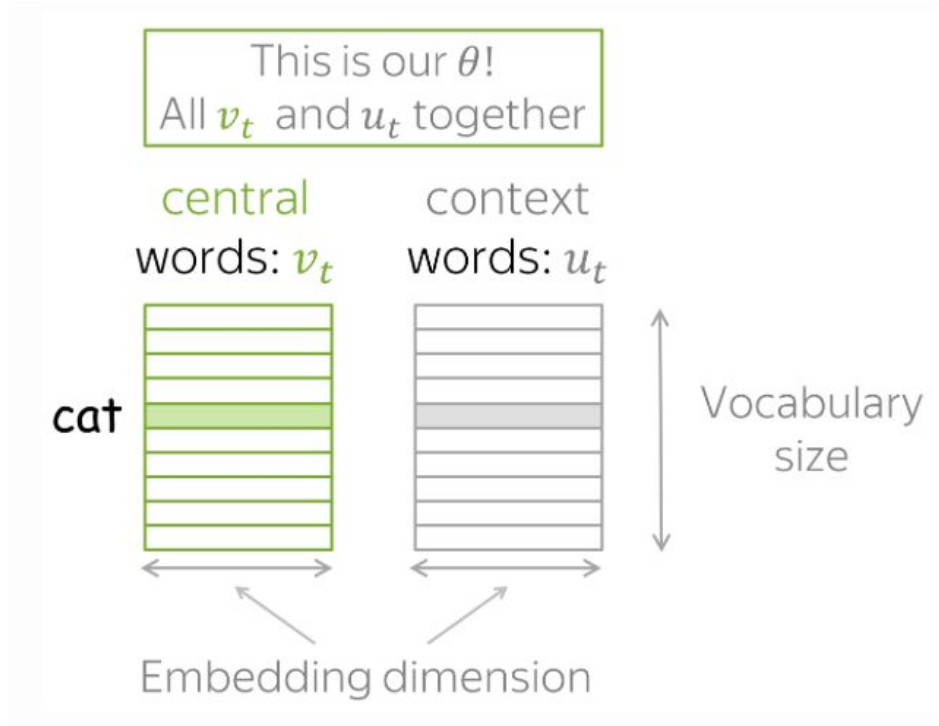
with a sliding
window



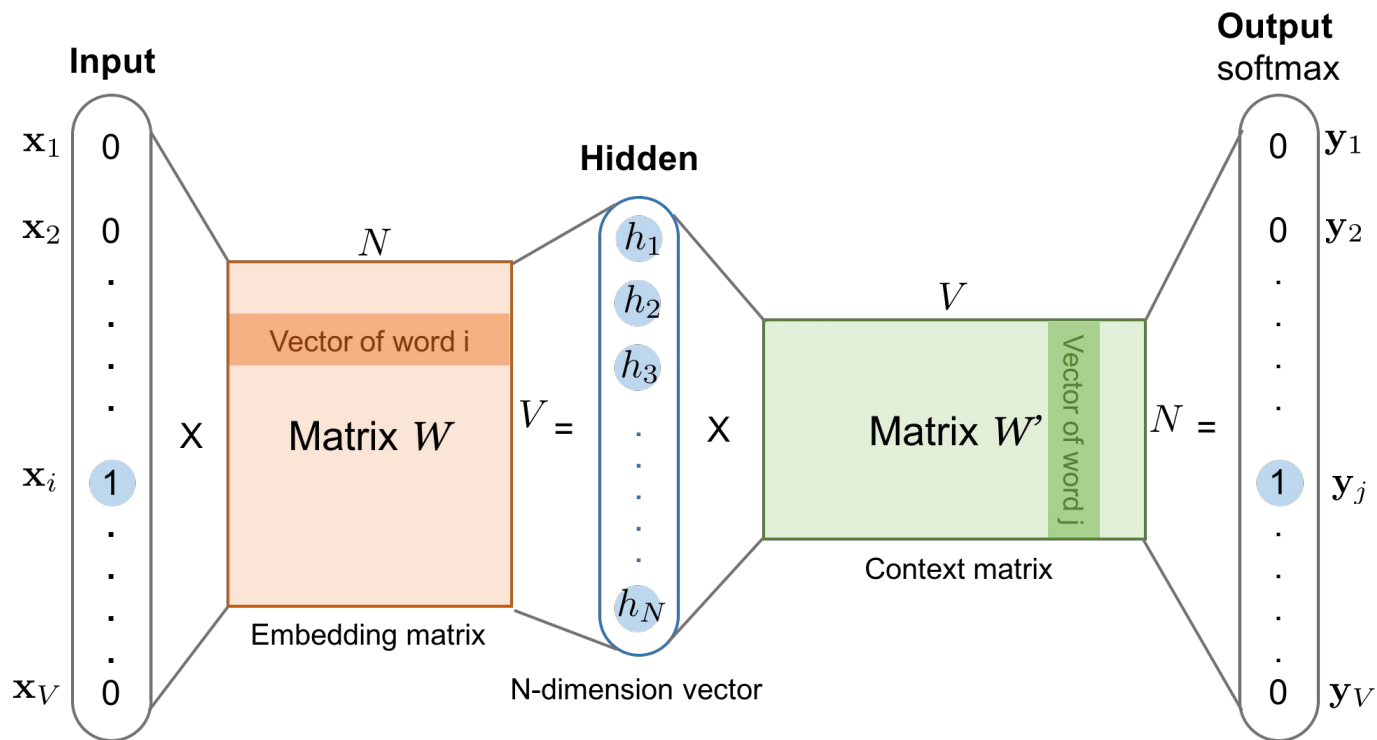
compute probability of the
context word given the central



Word2Vec



Word2Vec



Word2Vec

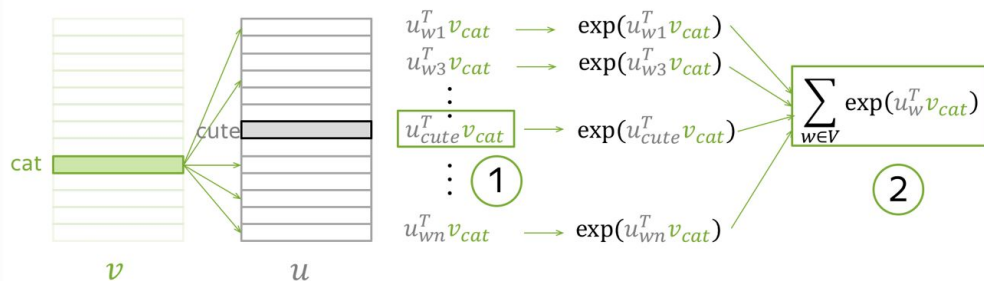
$$P(o|c) = \frac{\exp(u_o^T v_c)}{\sum_{w \in V} \exp(u_w^T v_c)}$$

Dot product: measures similarity of o and c
Larger dot product = larger probability

Normalize over entire vocabulary
to get probability distribution

Word2Vec

1. Take dot product of v_{cat} with **all** u
2. exp
3. sum all

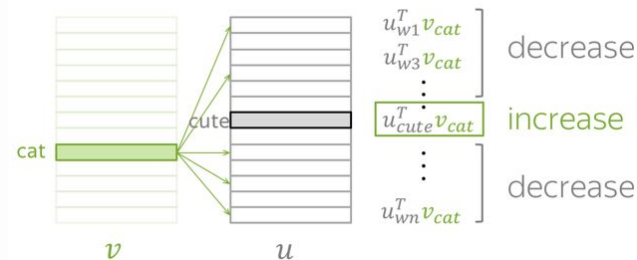


4. get loss (for this one step)
5. evaluate the gradient, make an update

$$J_{t,j}(\theta) = \underbrace{-u_{cute}^T v_{cat}}_{\text{①}} + \log \underbrace{\sum_{w \in V} \exp(u_w^T v_{cat})}_{\text{②}}$$

$$v_{cat} := v_{cat} - \alpha \frac{\partial J_{t,j}(\theta)}{\partial v_{cat}}$$

$$u_w := u_w - \alpha \frac{\partial J_{t,j}(\theta)}{\partial u_w} \quad \forall w \in V$$



Negative Sampling

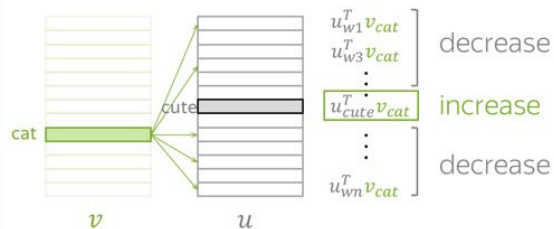
Dot product of v_{cat} :

- with u_{cute} - increase,
- with all other u - decrease



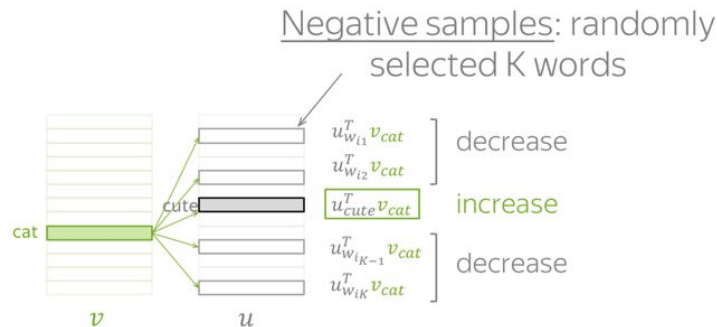
Dot product of v_{cat} :

- with u_{cute} - increase,
- with a subset of other u - decrease



Parameters to be updated:

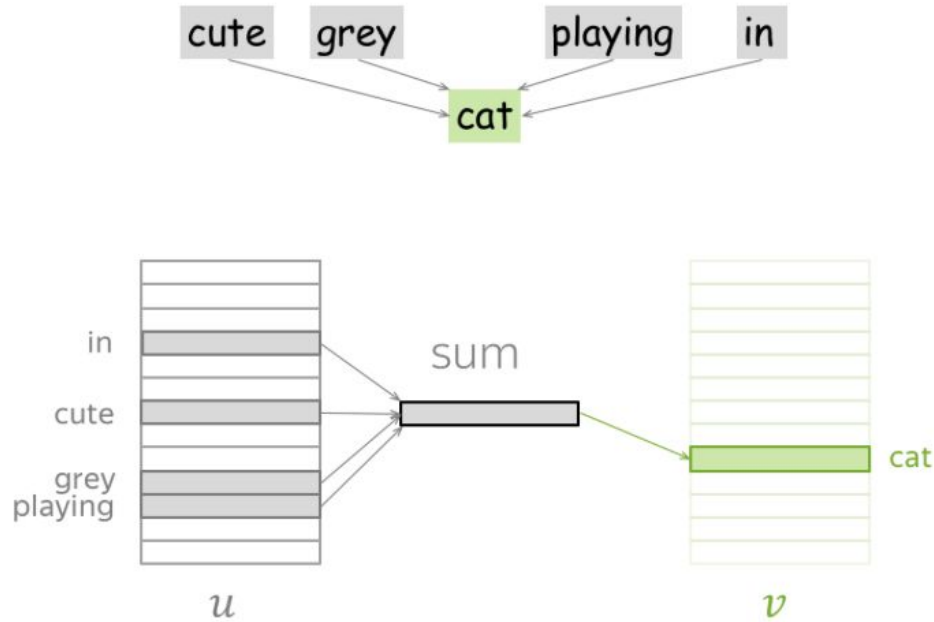
- v_{cat}
- u_w for all w in the vocabulary $|V| + 1$ vectors



Parameters to be updated:

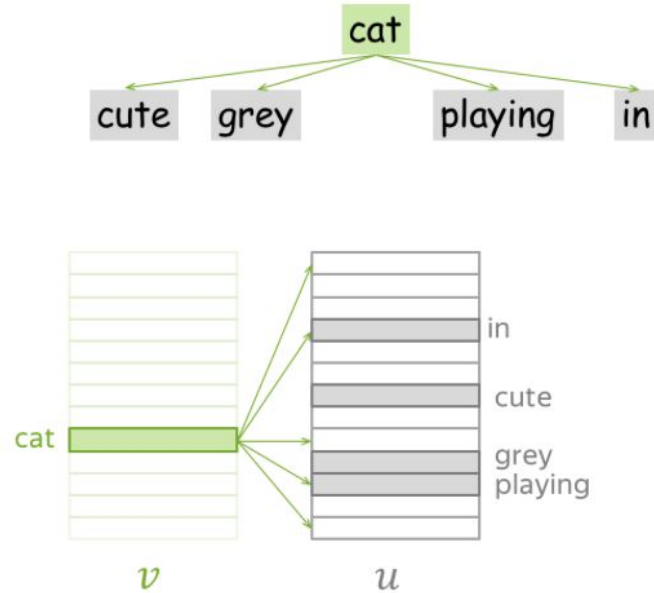
- v_{cat}
- u_{cute} and u_w for w in K negative examples $K + 2$ vectors

Continuous Bag of Words aka CBOW



CBOW: from sum of context predict central

Skip-gram

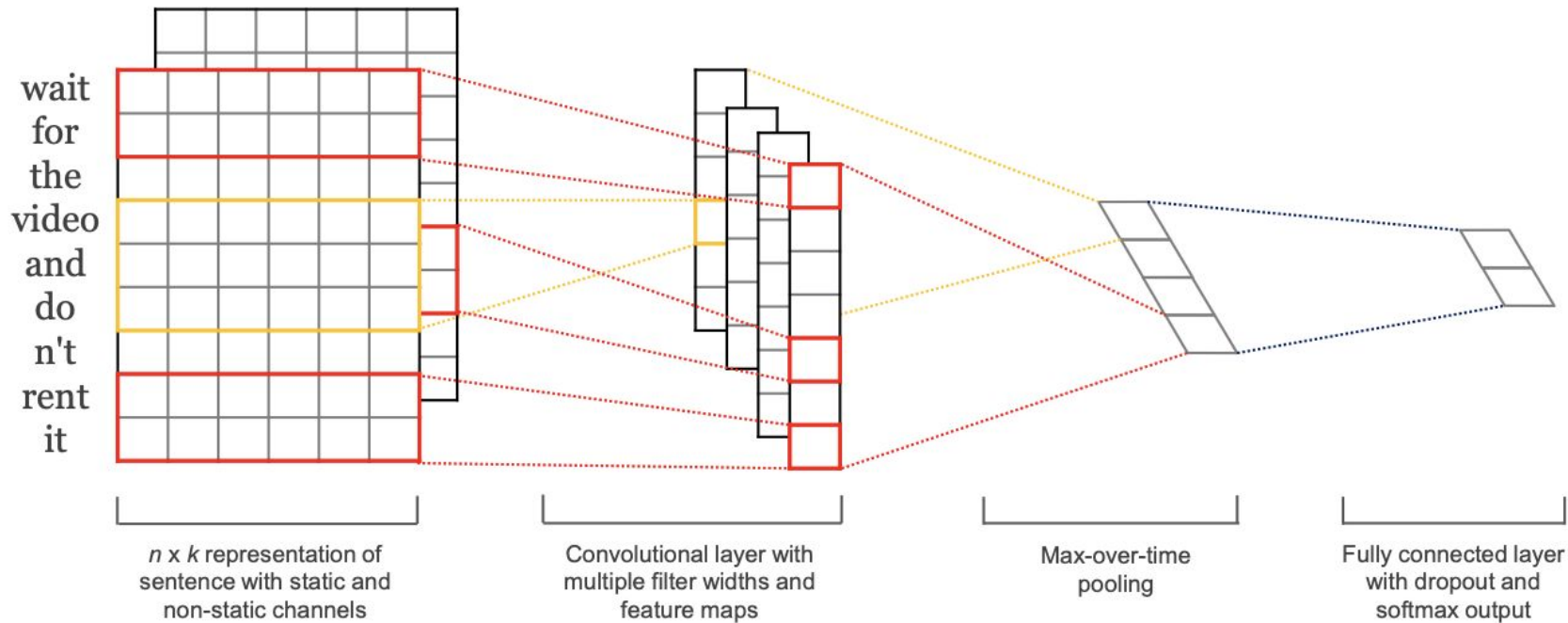


Skip-Gram: from **central** predict context
(one at a time)

FastText



Conv for text

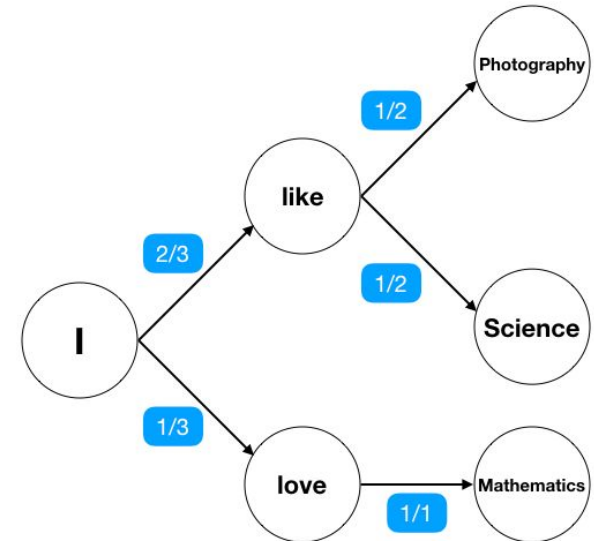


Why noone uses it?

- 1) Receptive field is not quite the same
- 2) How we usually read? Is it the same?
- 3)

Markov models

- 1) The presence of word W is determined by k words before it
- 2) $p(w_1, \dots, w_n) = p(w_1)p(w_2|w_1) \dots p(w_n|w_{n-1}, \dots, w_{n-k})$
- 3) Let's estimate $p(W)$
- 4) Profit



How can we make it better?

RNN

Transformer